# CS & IT ENGINEERING




## Algorithms

## Dynamic Programming




Lecture No.- 02

By- Aditya sir

# Recap of Previous Lecture



Topic

Topic

Dijkstra's SSSP Algo

# Topics to be Covered



- Topic
- Topic — PYQs
- Topic — Intro to DP



# About Aditya Jain sir

1. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
2. Represented college as the first Google DSC Ambassador.
3. The only student from the batch to secure an internship at Amazon. (9+ CGPA)
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program, specialization in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10
9. Joined Dream11 as a Data Scientist
10. Have mentored 12,000+ students & working professions in field of Data Science and Analytics
11. Have been mentoring & teaching GATE aspirants to secure a great rank in limited time
12. Have got around 27.5K followers on Linkedin where I share my insights and guide students and professionals.




Telegram Link for Aditya Jain sir: https://t.me/AdityaSir_PW

Tomorrow : 8 - 10:30 AM

→ Greedy DPP → Recording → [English]

→ 10th June (Hinglish)



# Topic : Dynamic Programming

```
1 .   Algorithm ShortestPaths (v, cost, dist, n)
2 .   // dist [j], 1 ≤ j ≤ n, is set to the length of the shortest
3 .   // path from vertex v to vertex j in a digraph G with n
4 .   // vertices. dist[v] is set to zero.- G is represented by its
5 .   // cost adjacency matrix cost[1: n, 1: n].
6 .         {
7 .         for i := 1 to n do
8 .         {
                  // Initialize S.
9 .               S[i] := false; dist[t] := cost[v, i];
10.   }
11.         S[v] := true; dist[v] := 0.0; // Put v in S.
12    .     for num := 2 to n - 1 do
```



# Topic : Dynamic Programming

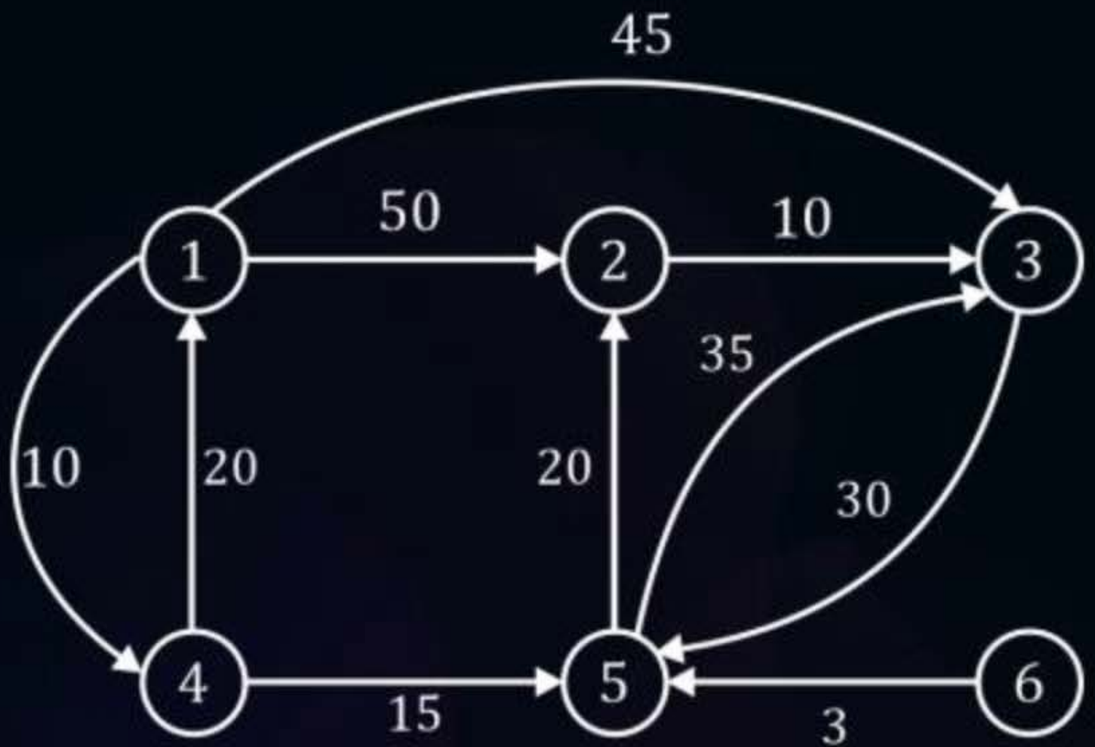
```
13    {
14          // Determine n - 1 paths from v.
15    Choose u from among those vertices not
16    in S such that dist[u] is minimum;
17    S[u] := true; // Put u in S.
18    for (each w adjacent to u with S[w] = false) do
19    // Update distances.
20    if (dist [w] > dist[u] + cost[u, w])) then
21    dist[w]:= dist[u] + cost[u, w];
22          }
23    }
```

Relaxation Process.



# Topic : Dynamic Programming

Source




| Set:s | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| {1} | 0 | 50 | 45 | 10 | ∞ | ∞ |
| {1,4} | 0 | - | - | 10 | - | O(n) |
| - | - | - | - | - | - | - |
| - | - | - | - | - | - | - |
| - | - | - | - | - | - | - |



# Topic : Dynamic Programming

Time Complexity analysis:-

1. Non-heap based implementation

   TC→ $O(n^2)$

2. Heap based implementation

   TC→ (n+e) logn



# Topic : Dynamic Programming

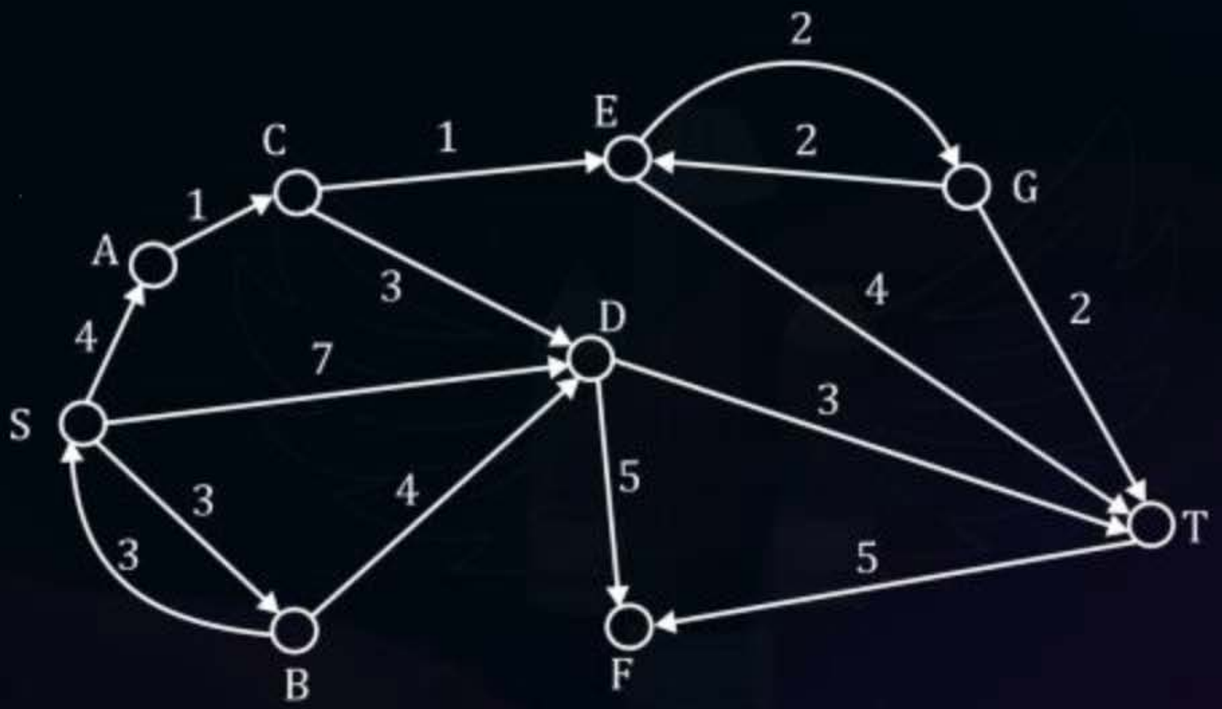
#Q. Applying Dijkstra's Algoirthm over the given Graph, which path is reported for 'S' to 'T';

A SBDT→ ~~35.5%~~

B SDT→ ~~28%~~

C SACEGT→ ~~23%~~

D SACET→ ~~14.7%~~



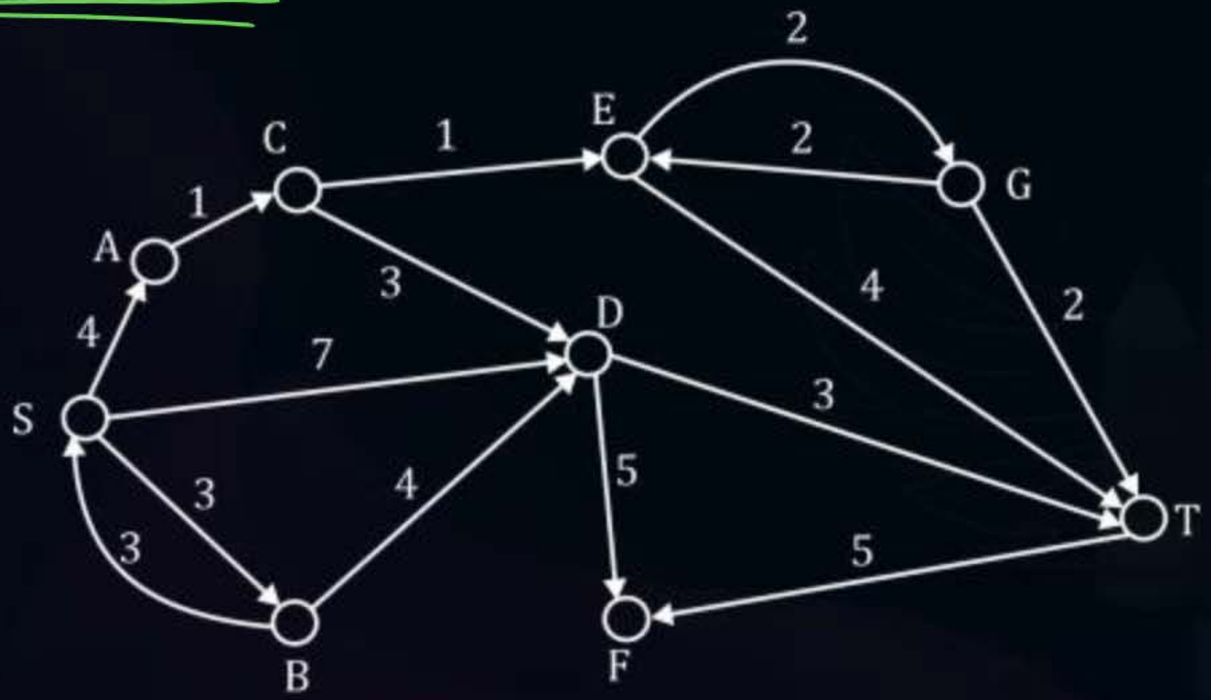


# Topic : Dynamic Programming

## Applying Dijkstra's SSSP Sapnning Tree approach

Spanning: Path reported by DJ SSSP is

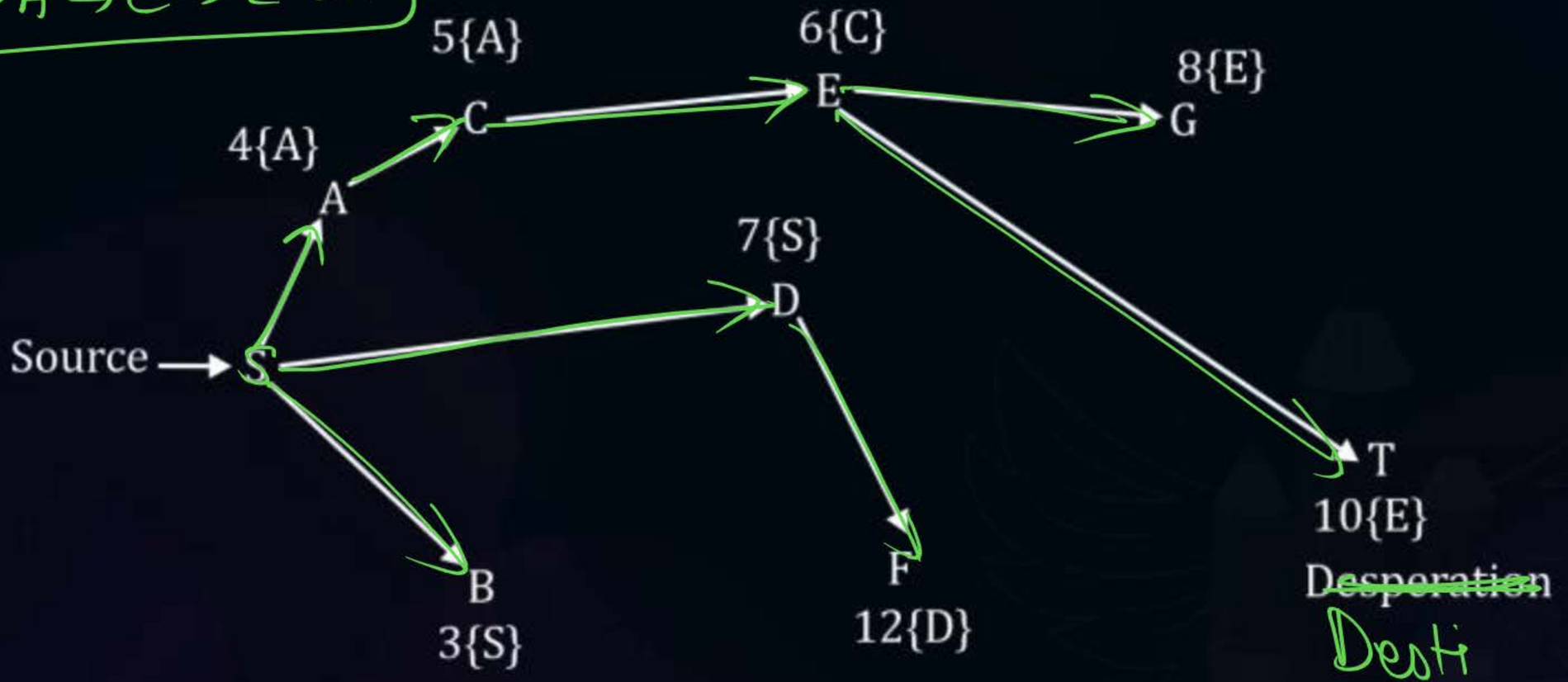
Only S→A → C → E → T





# Topic : Dynamic Programming

S → A → C → E → T




Final Spanning Tree By DJ SSSP.

Relaxation happen only when a strictly lesser path cost is found.



# Topic : Dynamic Programming

P. 49

#Q. Let G be weighted connected undirected graph with distinct positive edge weights. If every edge weight is increased by the same value, then which of the following statements is/are true?

$S_1$. Minimum spanning Tree of the graph does not change.

$S_2$. Shortest path between any pair of vertices does not change

37%

A. Only $S_1$ true

V same in Gate'25

B. Only $S_2$ true

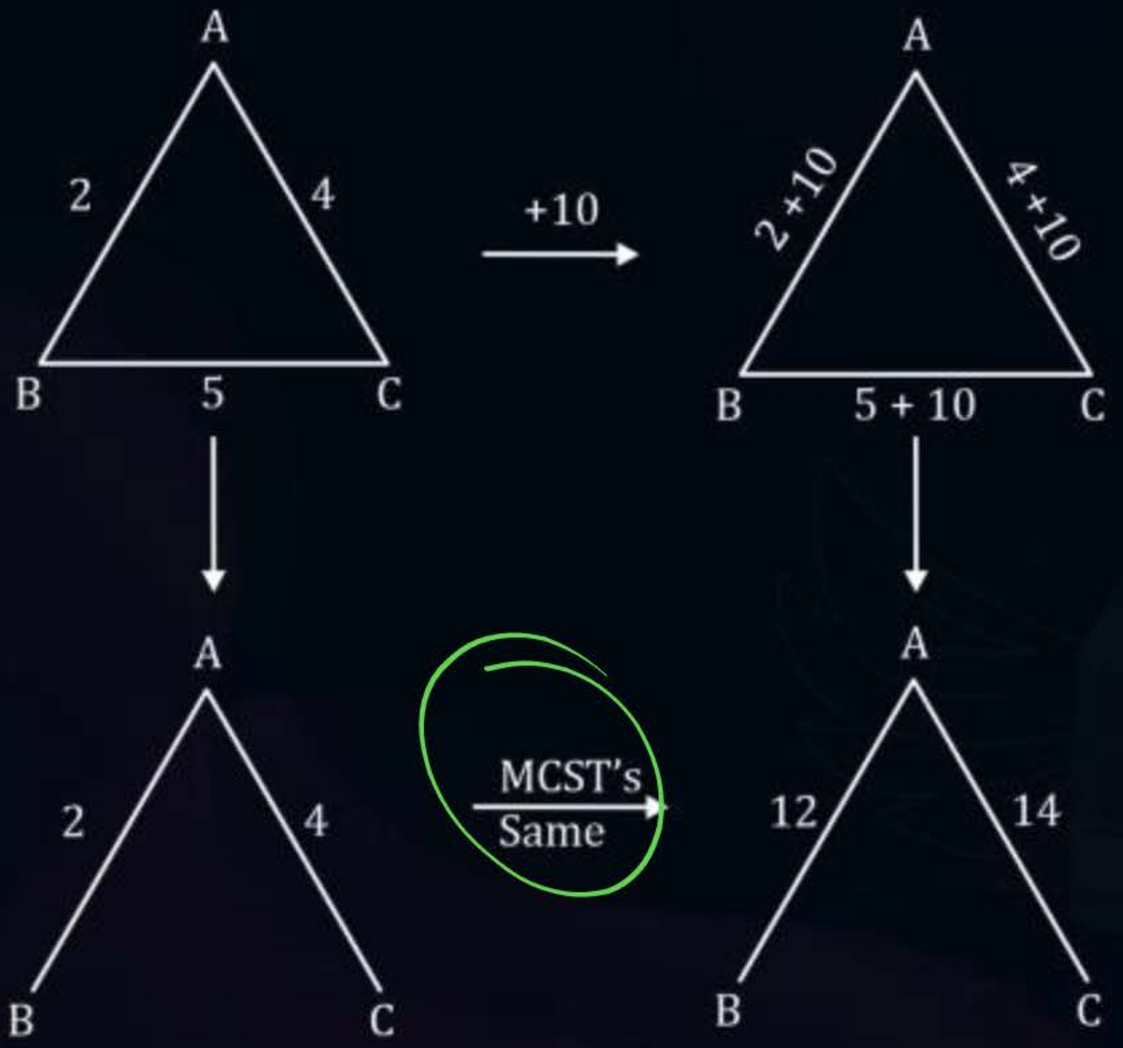
C. Both $S_1$ and $S_2$ true

D. Both $S_1$ and $S_2$ are False



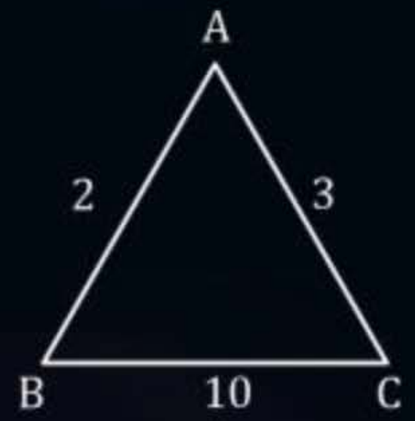
# Topic : Dynamic Programming

S2:

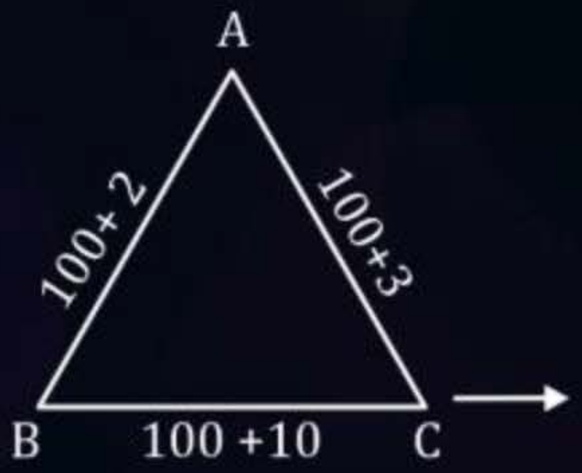


# Topic : Dynamic Programming

S2:




+ 100




Shortest Path from B→ C ?

B → A → C = 2 + 3 = 5 (BAC)

2 3

BAC

BC

B —110→ C

BAC → 205
BC → 110



# Topic : Dynamic Programming

#Q. Let G = (V, E) be any connected undirected edge- weighted graph. The weights of the edges in E are positive and distinct. Consider the following statements:

I. Minimum Spanning Tree of G is always unique

II. Shortest path between any two vertices of G is always unique.

Which of the above statements is/are necessarily true?

58%

A. I only

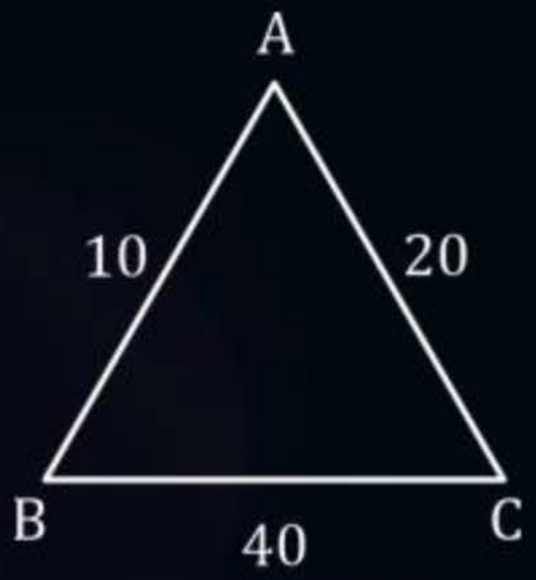
B. II only

C. Both I and II

D. Neither I and II

# Topic : Dynamic Programming

**Eg.1.**




B → C

B → A → C

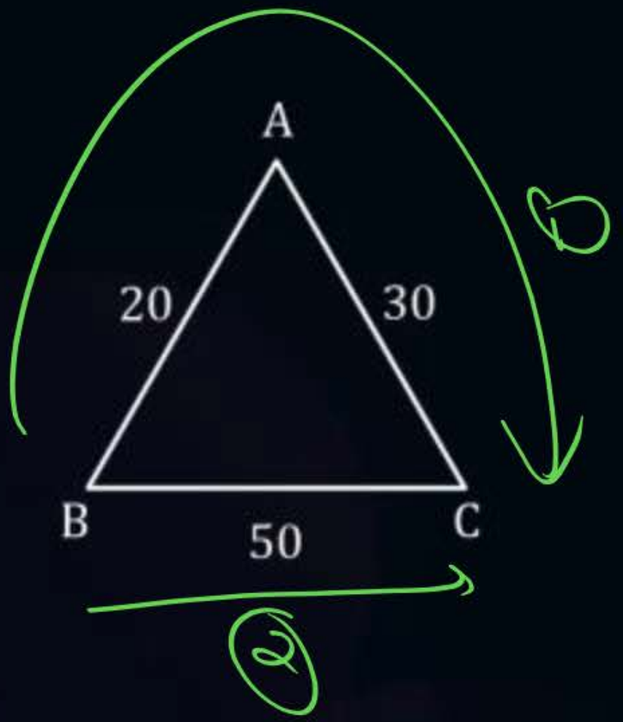
10 20

10 + 20 = 30



# Topic : Dynamic Programming

**Eg.2.**




Shortest Path

B ⟶ C

(1) B ⟶ A ⟶ C
20 30

20 + 30 = 50

(2) B ⟶ C = 50

Shortest Path Is NOT unique in this case.



# Topic : Dynamic Programming

#Q. Consider the following table :

| Algorithm | Design Paradigms |
|---|---|
| (P) Kruskal | (i) Divide and Conquer |
| (Q) Quick sort | (ii) Greedy |
| (R) Floyd-Warshall | (iii) Dynamic Programming |

Match the algorithms to the design paradigms they are based on.

A (P) ↔ (ii), (Q) ↔ (iii), (R) ↔ (i)

B (P) ↔ (iii), (Q) ↔ (i), (R) ↔ (ii)

C (P) ↔ (ii), (Q) ↔ (i), (R) ↔ (iii)

D (P) ↔ (i), (Q) ↔ (ii), (R) ↔ (iii)

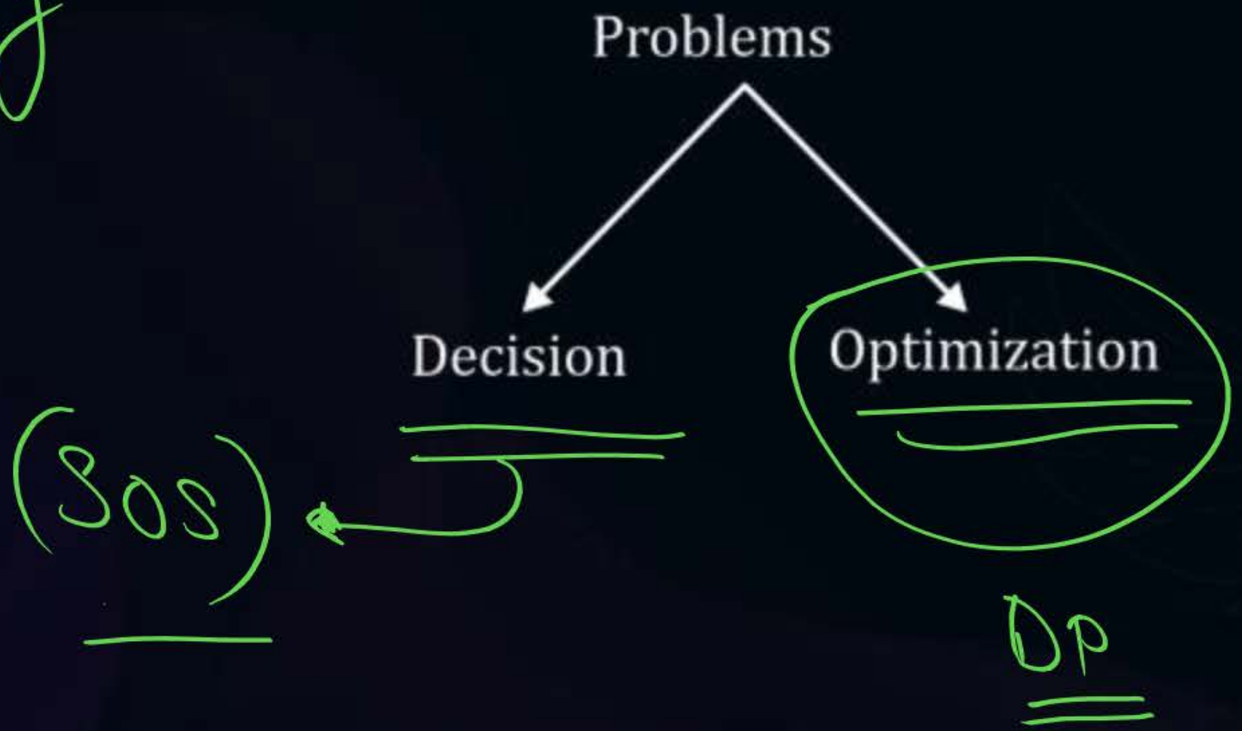


# Topic : Dynamic Programming

## Introduction to Dynamic Programming:

~~Sorting~~/ Tabulating the values or Results of the sub- Problems.

Storing

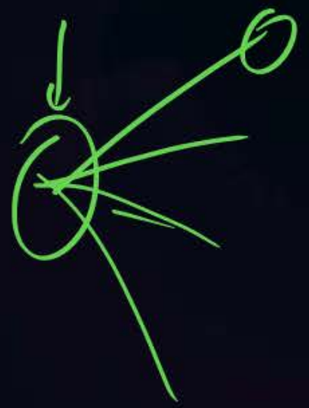




# Topic : Dynamic Programming

Dynamic Programming (DP) is an algorithm design method used for solving problems, whose solutions are viewed as a result of making a set / sequence of decisions.

- One way of making these decisions is to make them one at a time in a step-wise (sequential) step-by-step manner and never make an erroneous decision. This is true of all problems solvable by Greedy method.)
- For many other problems it is not possible to make step-wise decisions based on local information available at every step, In such a manner that the sequence of decision made is optimal.





# Topic : Dynamic Programming

**Eg.** 0/1 Knapsack




Main diff. between greedy & DP.

Sometimes greedy appr. Does not give optimal solution to some problems.



# Topic : Dynamic Programming

Consider the weights and values of times listed below: Note that there is only one unit of each item.

M = 11 (Capacity)

| Item number | Weight (in kgs) | Profit (in Rupees) |
|---|---|---|
| 1 | 10 | 60 |
| 2 | 7 | 28 |
| 3 | 4 | 20 |
| 4 | 2 | 24 |

Binary Knapsack



# Topic : Dynamic Programming

| | W | P |
|---|---|---|
| 01 | 10 | 60 |
| 02 | 7 | 28 |
| 03 | 4 | 20 |
| 04 | 2 | 24 |

$\rightarrow 60/10 = 6 \rightarrow 2 \times x_1 = 0$

$\rightarrow 27/7 = 4 \rightarrow 4 \times x_2 = 0$

$\rightarrow 40/4 = 5 \rightarrow 3 \times x_3 = 1$

$\rightarrow 24/2 = 12 \rightarrow 4 \times x_1 = 1$

M = 11

11- 2 =9

10 > 9 → Skip

9 – 4 = 5

Max profit by queued appr. To 0/1 knapsack

$= \sum_{i=1}^{4} pixi$

$= 60 \times 0 + 28 \times 0 + 20 \times 1 + 24 \times 1$

$= 20 + 24 = 44$

$= 44$



# Topic : Dynamic Programming

But optimal profit

when taken only object $O_1$

= 60

Hence, greedy approach fails to give optimal solution in 0/1 knapsack (may or not give)



# Topic : Dynamic Programming

**Eg.1.** Coin change Problem (DP vs Greedy)

Problem:- Given a set of coin values, construct a sum of money using as few coins as possible.

You can use each coin value any number of times.

Coin Values: $\{C_1, C_2 \ldots C_K\}$

Target Money (Sum) : N



# Topic : Dynamic Programming

**Eg.1.** Coins = {1,2,5}, Sum = N = 12

Greedy Method: 5 + 5 + 2 = 12

Number of coins required = 3

in this case, Greedy gives optimal assume.

eg2: {1, 3, 4}

N = 6

4 + 1 + 1 ⇒ 3 coins



# Topic : Dynamic Programming

**Eg.1.** Coins = {1, 3, 4 } , N = 6 (sum)

Greedy approach: 4 + 1+ 1 = 6 (Number of coins = 3)

in general, optimal sol. 3 + 3 = 6 (number of coins = 2)



# Topic : Dynamic Programming

In above example , greedy approach failed to give optimal solution.

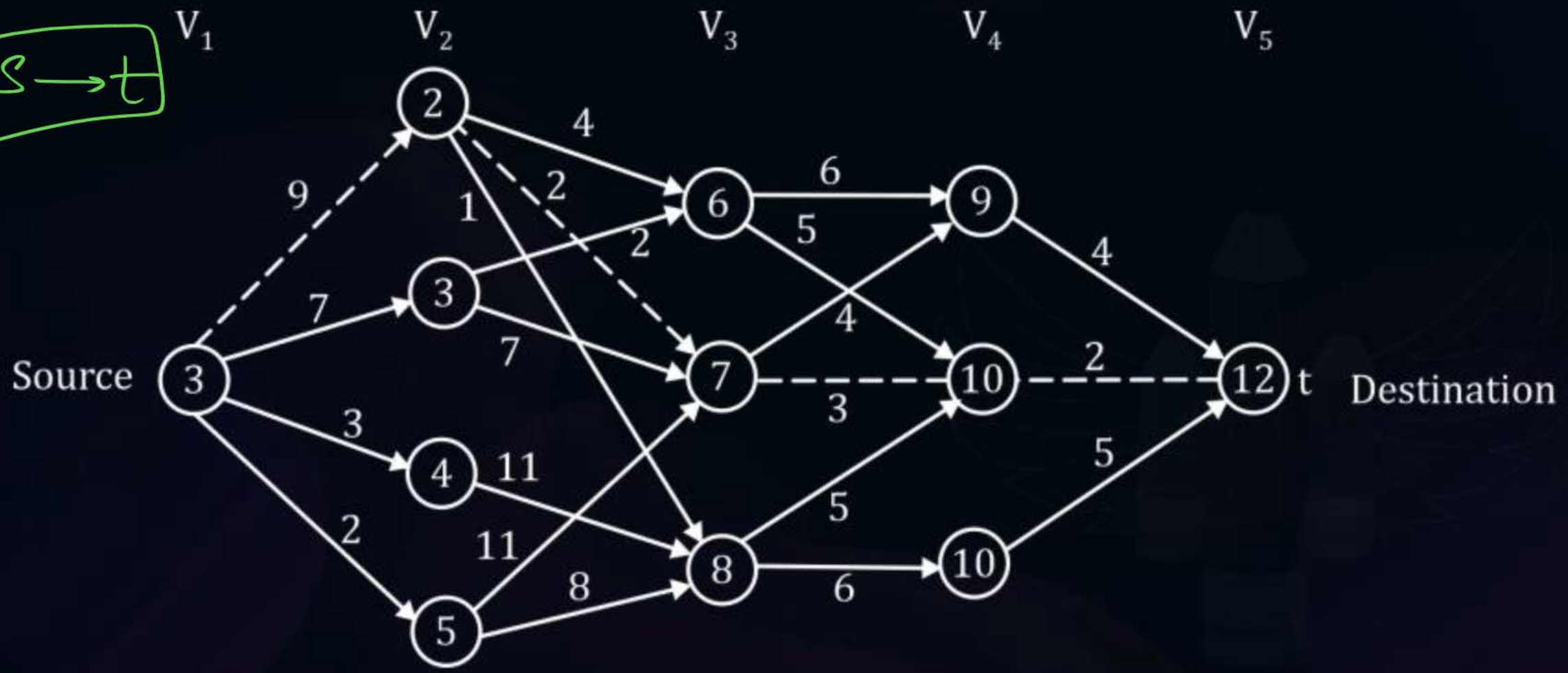
Hence, greedy approach doesn't guarantee (may or may not give) optimal solution to coins change problem.



# Topic : Dynamic Programming

Stages

$s \rightarrow t$



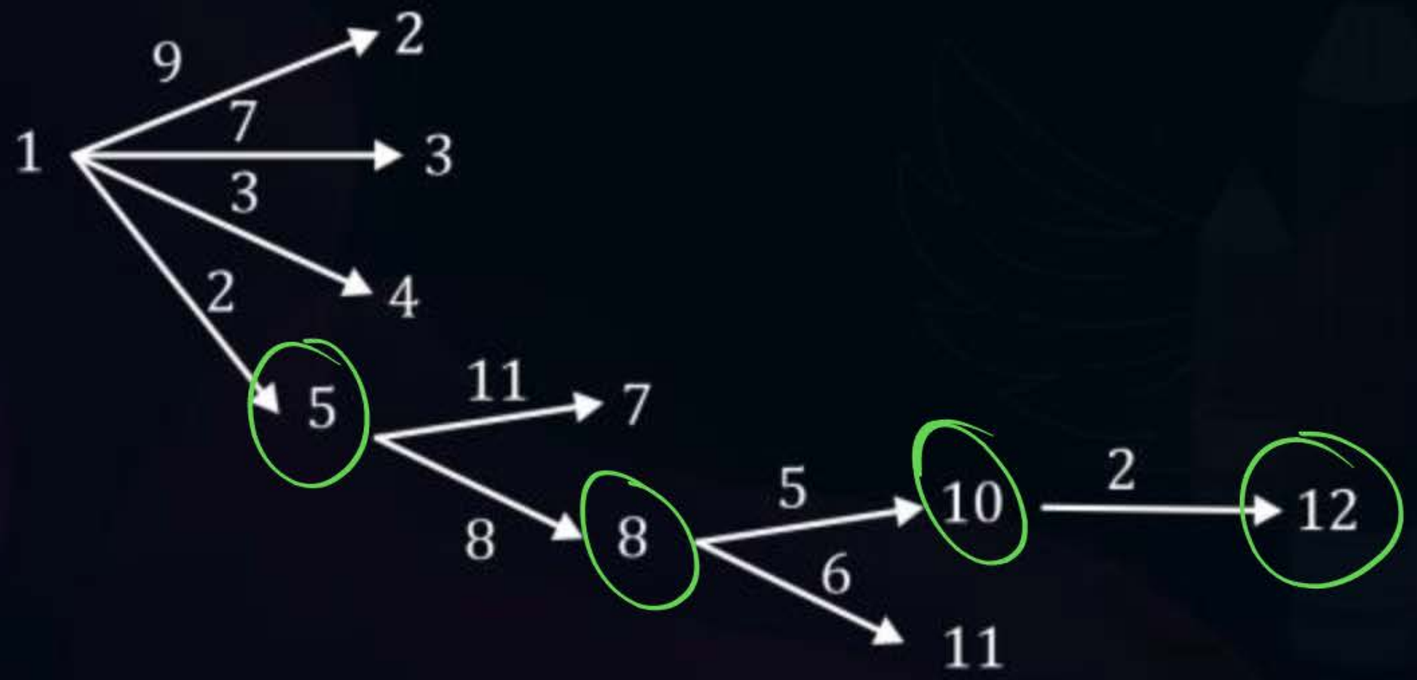


# Topic : Dynamic Programming

(s)1 → 12(t) using greedy approach.

Greedy approach :- 1 → 5 → 8 → 10 → 12

2 + 8 + 5 + 2 = 17 {not optimal}





# Topic : Dynamic Programming

But is 17 optimal /min path cost?

$< 1 \rightarrow 2 \rightarrow 7 \rightarrow 10 \rightarrow 12 >$

$9 + 2 + 3 + 2 = 16$

$< 1 \rightarrow 3 \rightarrow 6 \rightarrow 10 \rightarrow 12 >$

$7 + 2 + 5 + 2 = 16$

Actual minimum cost paths from $1 \rightarrow 12$

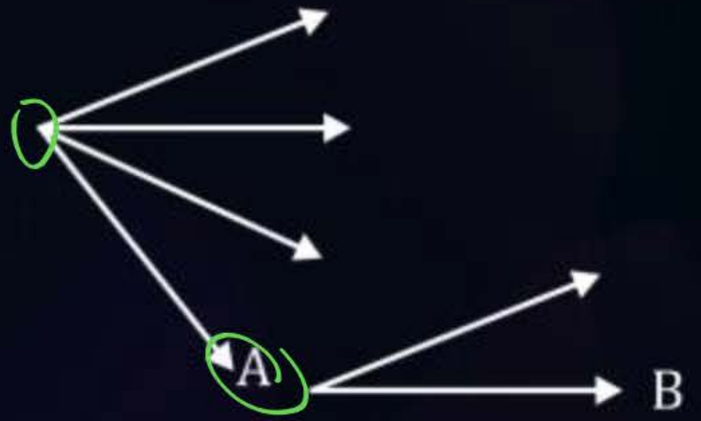
Hence, greedy approach of SPSP failed to give optimal cost here.

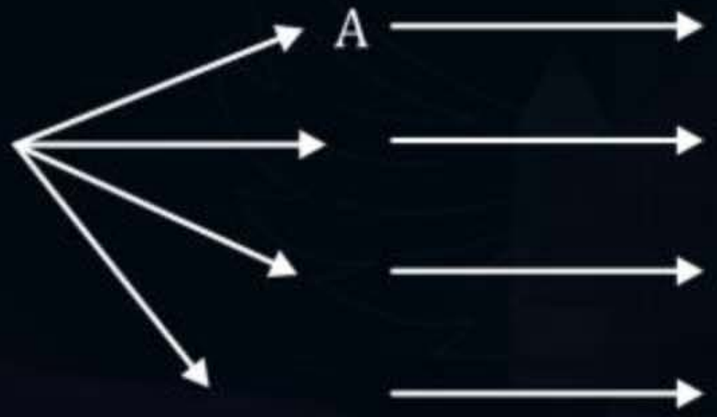


# Topic : Dynamic Programming

Local vs Global info

greedy Global





# Topic : Dynamic Programming: (DP)

DP vs greedy

1. 0/1 Knapsack
2. Coin change problem
3. Multi-stage graph

Greedy → Local optimisation

DP→ Global optimisation

DP gives optimal solution but not give optimal solution.

greedy may



# Topic : Dynamic Programming: (DP)

P → Path 1 → C1
Path 2 → C2
Path 3 → C3
Path 4 → C4
Path 5 → C5
Path n → Cn

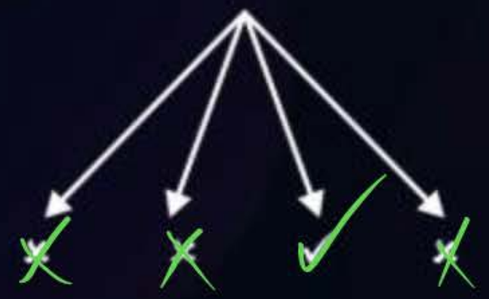
min

Min → Optimal solution

Enumeration **or** Brute force

Drawback → High Complexity

DP ☞ Brut force





# Topic : Dynamic Programming: (DP)

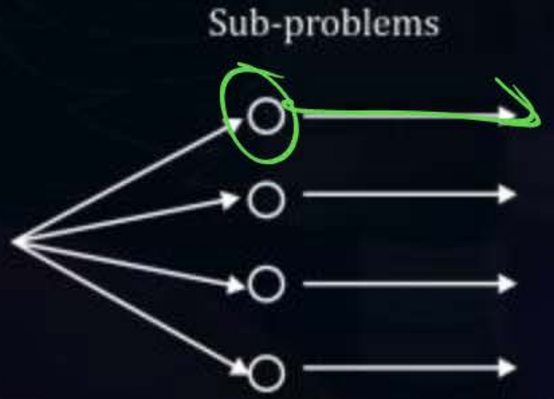
- One way of solving problem in which it is not possible to make sequence of decisions in step-wise manner leading to optimal solutions, is to enumerate all decision sequences (Brute force) & then pick up the best solution (optimal).
- But the drawback with brute force/ Enumeration is excessive Time/Space requirements
- Dynamic Programming (DP) based on Enumeration often tries to reduce the amount of enumeration by curtailing those decision sequence from where there is no possibility of getting optimal solution. (That's how it may bring down the time complexity)
- In Dynamic programming these set of optimal decisions are made by applying principal of optimality.



# Topic : Dynamic Programming: (DP)

- Principle of Optimality: states that whatever the initial state & decision are the remaining sequence of decisions must constitute an optimal decision sequence with regard to the state resulting from the first decision.
- Essential difference between Greedy method & Dynamic programming (DP) is that Greedy method always generates only one decision sequence. Whereas in DP enumeration many decision sequences can be generated.







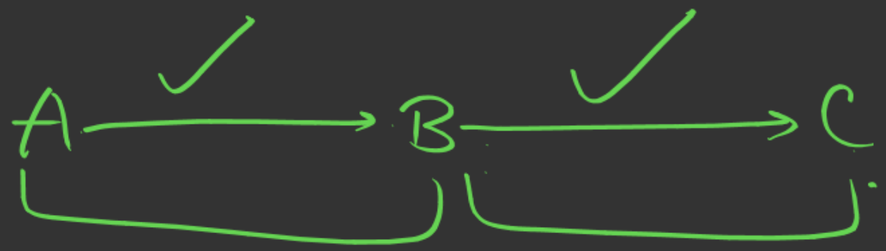
# Topic : Dynamic Programming: (DP)

(V.V.Imp)

- Another important feature of D.P is that optimal solutions of the sub problems are retained (cached/ stored in a table) to avoid recomputing their values. (Invariably this feature also leads to saving of time)

**D.P implementation**

- **Memoization (Top-Down Approach)** → Recursive
- **Tabulation (Bottom-Up)** → Iterative


A
B
C



# Topic : Dynamic Programming: (DP)

(i) **Splitting of original problems into Subproblems:** Be able to split the original problem into subproblems in a recursive manner (so that the Subproblems can be further divided into sub-subproblems). This process of splitting should continue till the Subproblems becomes small.

(ii) **Subproblems Optimality (Optimal Substructure):** An optimal solution to the problem must result from optimal solutions to the subproblems with combine operation

(iii) **Overlapping Subproblems:** Many subproblems themselves contain common sub-subproblems. Therefore, it is desirable to solve the small problem & store/cache their results, so that they can be used in other subproblems.



# Topic : Dynamic Programming: (DP)

## Fibonacci Number:

Fib series: 0,1,1,2,3,5,8,13,21,34.

Fib(n) = Fib (n-1) + Fib (n-2), n > 1

Fib(n) =1, n = 1

Fib (n) = 0, n = 0



# Topic : Dynamic Programming: (DP)

## Normal Recursive Implementation of Fib (n):

```
Algo Fib (int n)
{
        If (n≤1) return (n);
        else
        {
              return (Fib (n-1)+ Fib (n-2));
        }
}
```



# Topic : Dynamic Programming: (DP)

## Time complexity Recurrence

$T(n) = C, n = 1$

$T(n) = T(n-1) + T(n-2) + a , n > 1$

We know,

$T(n-2) < T(n-1)$

$T(n) = T(n-1) + T(n-1) + a, \ n > 1$

$T(n) = 2T(n-1) + a , n > 1$

$T(n-1) = 2T(n-2) + a$



# Topic : Dynamic Programming: (DP)

$T(n) = 2[2T(n-2) + a] + a$

$= 2^2 T(n-2) + 2a + a \ldots\ldots(2)$

$T(n-2) = 2T(n-3) + a$

$T(n) = 2^2 (2T(n-3) + a) + 2a + a$

$= 2^3 T(n-3) + (2^2 * a \; 2^1 a + 2^0 a)$

$2^0 + 2^1 + 2^2 = 2^3 - 1$

General Term

$T(n) = 2^k T(n-k) + (2^k - 1) * a$

$n - k = 1 \rightarrow k = (n-1)$



# Topic : Dynamic Programming: (DP)

$$T(n) = 2^{(n-1)} * T(1) + (2^{(n-1)} - 1) * a$$

$$= c * \frac{2^n}{2} + \left(\frac{2^n}{2} - 1\right) * a$$

$$T(n) \rightarrow O(2^n)$$

↓

Normal Fibo



# Topic : Dynamic Programming: (DP)

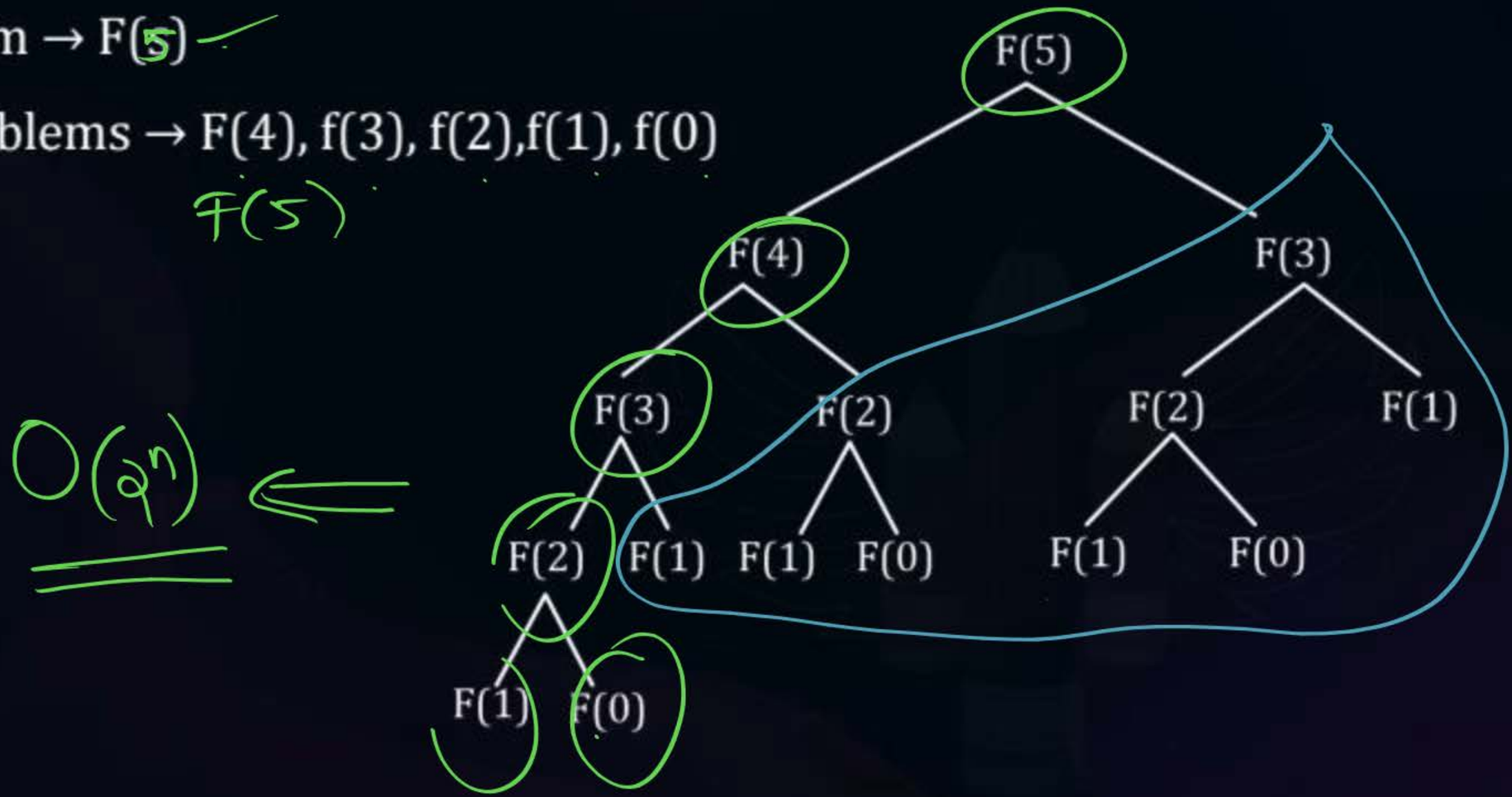
Normal Recursive Approach: Tree of function calls

Original problem → F(5)

unique Sub-problems → F(4), f(3), f(2),f(1), f(0)

(Q) Find fibo (5):

F(5)




O(2^n) ⇐



# Topic : Dynamic Programming: (DP)

## 1 Top-down- Memoized implantation of Fib (n)

```
Algo memiFb(n)
{
        if (m [n] is undefined)
        {
            if (n <=1) result = n;
            else
            result = memFib (n-1) + memFib(n-2);
            m[n] = result; //memorizing (caching)
        }
        return (m[n]);
}
```

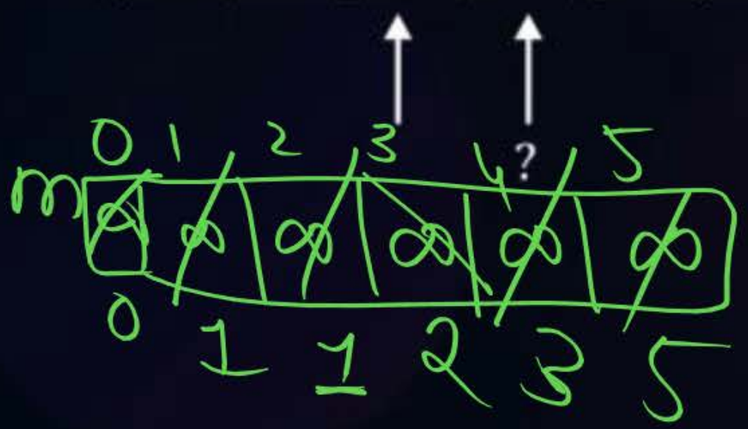
# Topic : Dynamic Programming: (DP)

## Top-Down code walkthrough:

#Q. Fib (5) = ?

$0 \rightarrow 5$

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| M: | 0 | 1 | 1 | 2 | 3 | 5 |




memFib(5): res: 3 + 2 = 5→ m [5]= 5

memFib(4):res: 2 + 1 = 3→ m [4]= 3

memFib(3):res: 1 + 1 = 2→ m [3]= 2

memFib(2): res: 1 + 0 = 1→ m [2]= 1

memFib(1):m [1]= 1

memFib(0): m [0]= 0

memFib(1): 1

memFib(2): m[2]

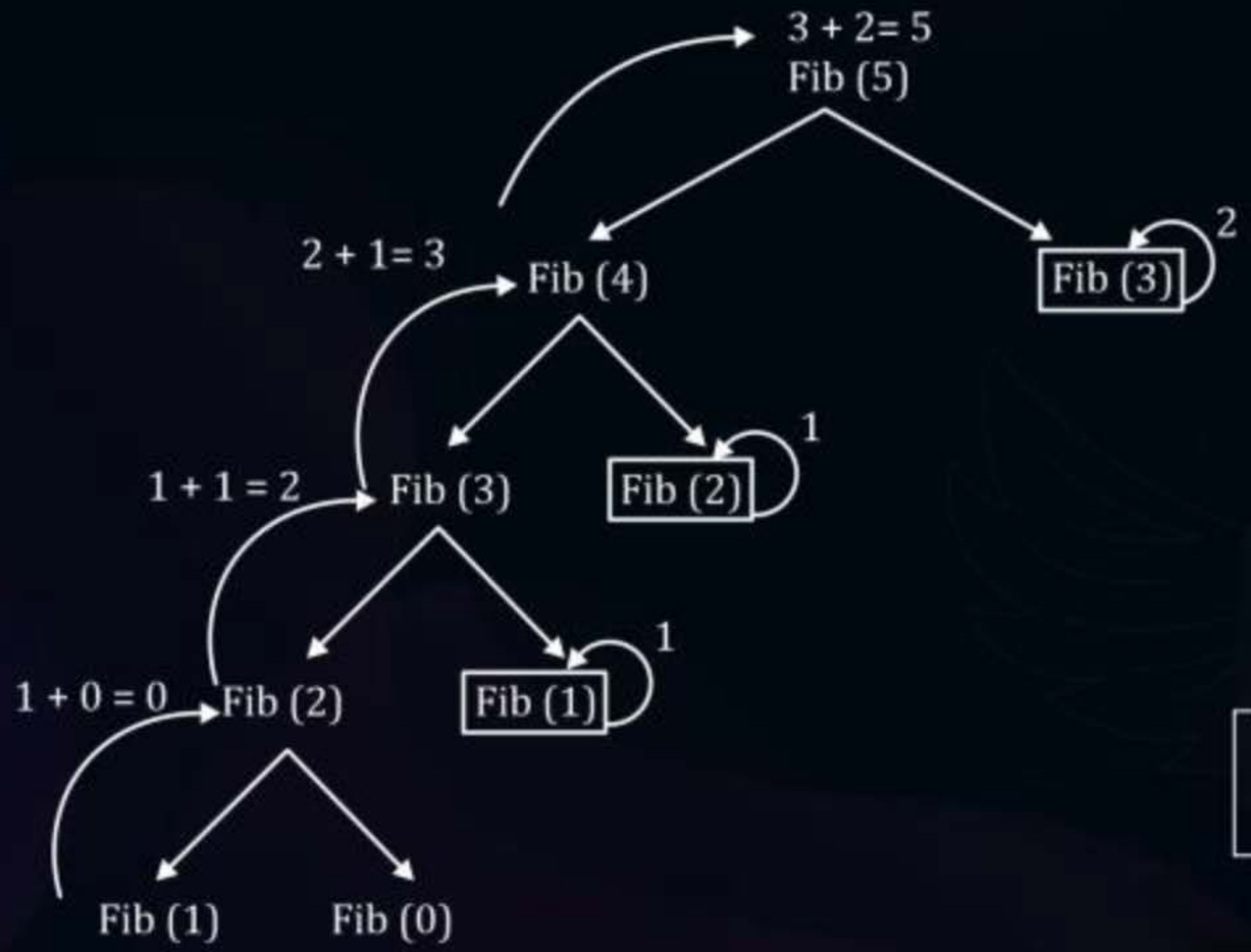
memFib(3): m [3]



# Topic : Dynamic Programming: (DP)

Function call Tree for above approach.

H.W




TC → O(n)

$O(2^n) \rightarrow O(n)$



# 2 mins Summary

- Topic
- Topic
- Topic
- Topic
- Topic

Dijktaa

PYQs

DP



# THANK - YOU